Ollscoil Teicneolaíochta an Oirdheiscirt
South East Technological University

THE EVOLUTION OF MACHINE LEARNING AND AI · WEEK 7 OF 12

Large Language Models

How they work, where they strain, and your first project step

Live lecture · 2 hours · Google Colab walkthrough · lab at home: 1 hour

THREE PARTS · TWO HOURS

Today

1 · The topic

Large language models: how they work, and where they are hitting limits.

2 · Your project

How today’s topic connects to your project, and this week’s step.

3 · Lab walkthrough

I do this week’s step live in Google Colab, every click.

KEY IDEA

At home (1 hour at most): Part A tries a language model on your own laptop; Part B does this week’s project step on your own law.

TWO HOURS, NO BREAK

Today’s route

Part 1	Large language models	⏱ 60 min
Part 2	How it connects to your project	⏱ 30 min
Part 3	Lab walkthrough in Google Colab	⏱ 20 min
Wrap-up	Your lab at home, and questions	⏱ 10 min

EVERYDAY ANALOGY

The class example all day is a **sourdough starter rising in a jar**. It is not one of the twelve project ideas, so I never do anyone’s project for them.

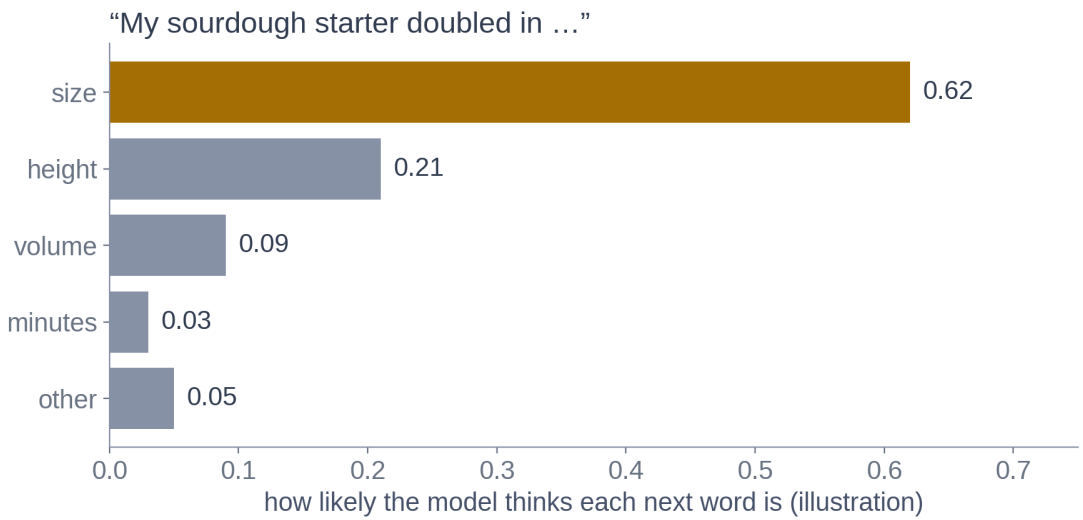
PART 1

Large language models

How they work, and where they strain

ONE SKILL: GUESS THE NEXT WORD

What a large language model does

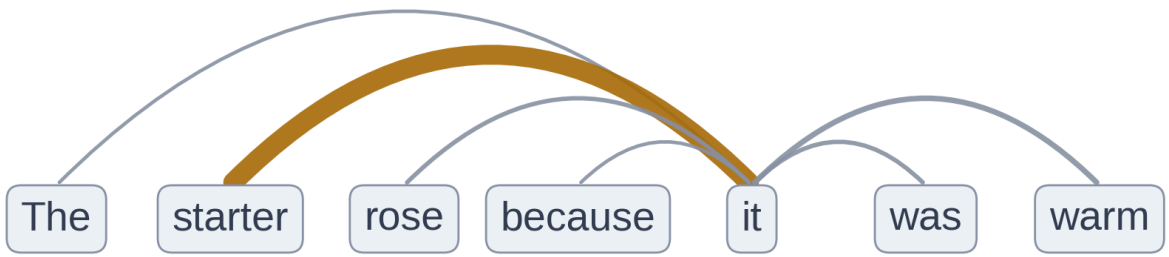


KEY IDEA

An LLM reads some text and guesses the most likely **next word**. Everything it does, answering, summarising, writing code, comes from that one skill, done extremely well.

A REMINDER FROM WEEK 4

What is inside: a transformer



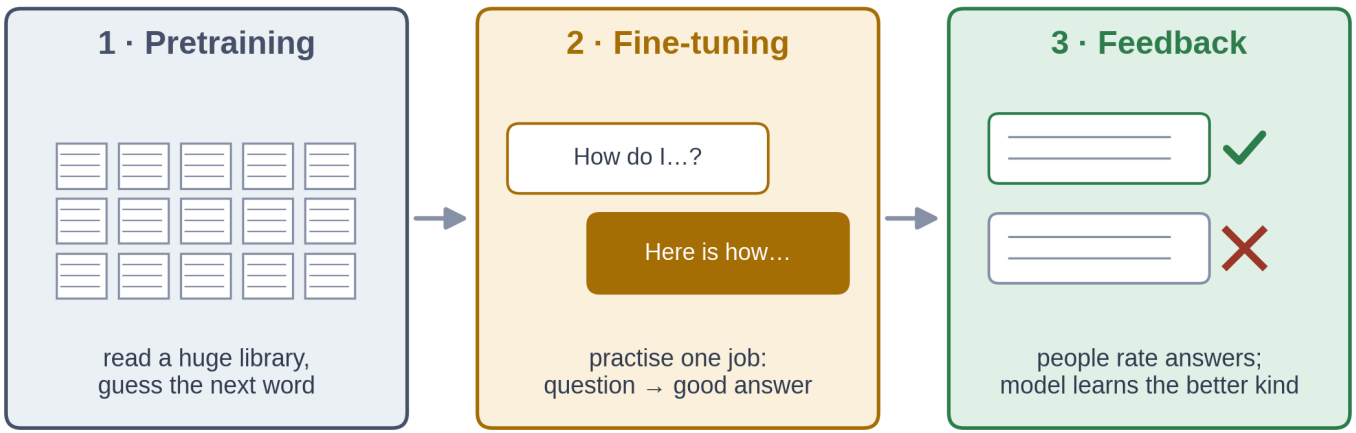
the word “it” pays most attention to “starter” (illustration)

KEY IDEA

Inside every LLM is a **transformer**. Its **attention** step lets every word look at every other word and decide which ones matter. That is how it keeps track of long sentences.

LIKE TRAINING AN APPRENTICE

How an LLM is made: three stages

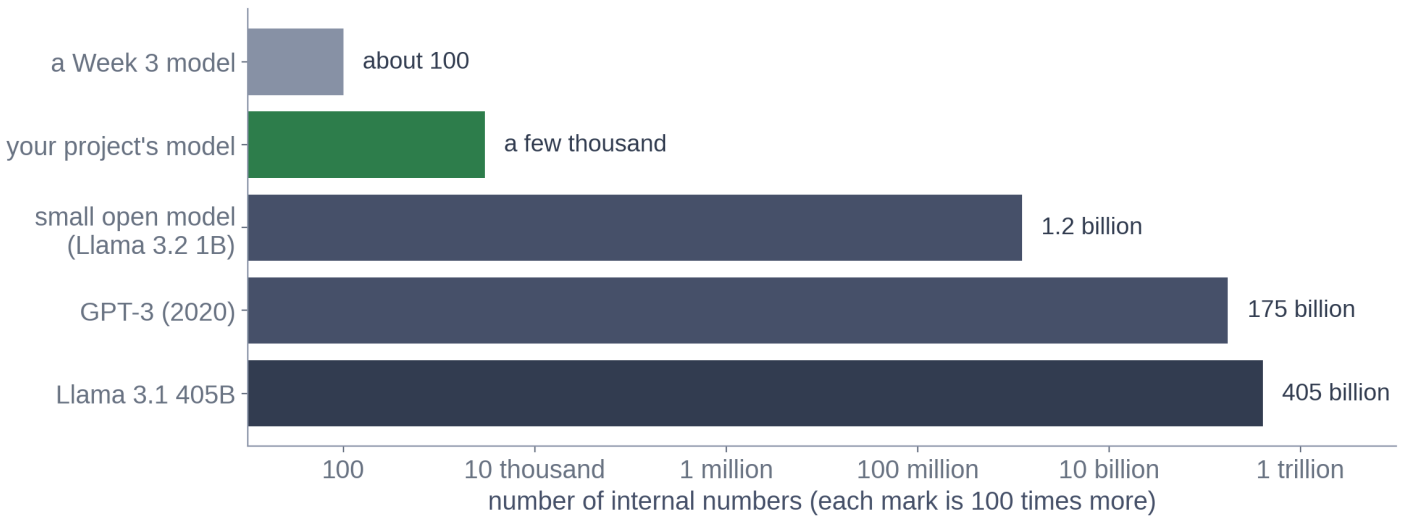


EVERYDAY ANALOGY

An apprentice chef first reads every cookbook in the library, then trains in one kitchen, then learns from what the customers send back.

WHERE THE KNOWLEDGE AND THE SIZE COME FROM

Stage 1, pretraining, is the expensive one



KEY IDEA

Pretraining reads **trillions** of words and tunes **billions** of internal numbers, for weeks on thousands of chips. It is where the model picks up grammar, facts and styles, and where almost all the cost goes.

THE MODEL NEVER SEES WHOLE WORDS

Tokens: how text is cut up

My sour dough starter doubled over night !

An illustration. At home, you will see the real split in the Tokeniser.

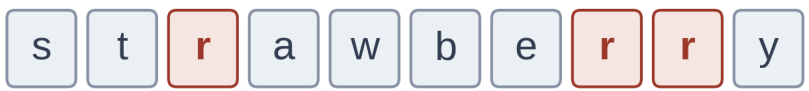
KEY IDEA

Before the model reads anything, the text is cut into pieces called **tokens**: whole words, parts of words, spaces, punctuation. In English, a token is about three-quarters of a word on average.

COST, LENGTH, AND SOME ODD MISTAKES

Why tokens matter

What you see:



What the model sees:



three chunks, no separate letters (an illustration of the idea)

Cost and speed

Companies charge per token, and longer text takes longer.

A length limit

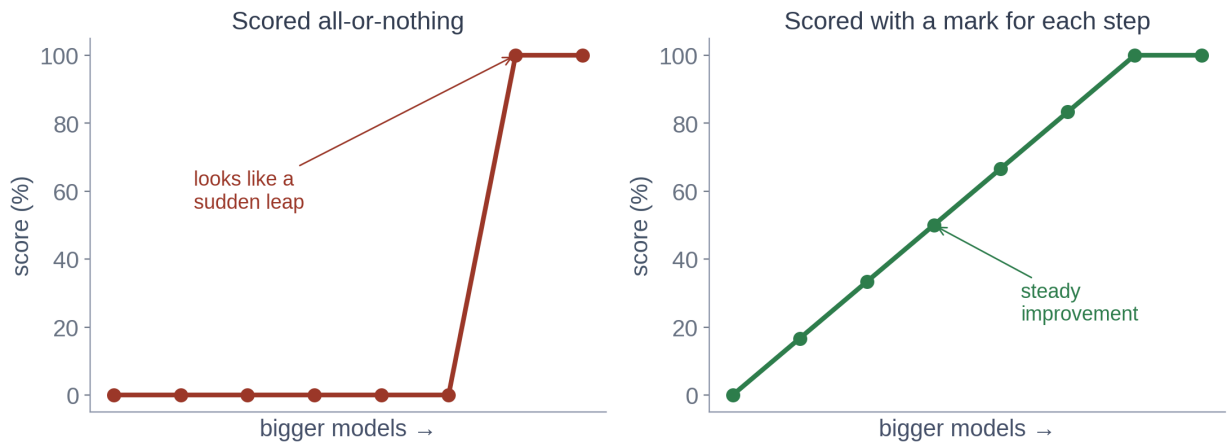
Each model can only read so many tokens at once.

Odd mistakes

Asked how many r’s are in “straw-berry”, models have often got it wrong.

SKILLS THAT SEEM TO APPEAR WITH SIZE

Emergent abilities, and the argument about them



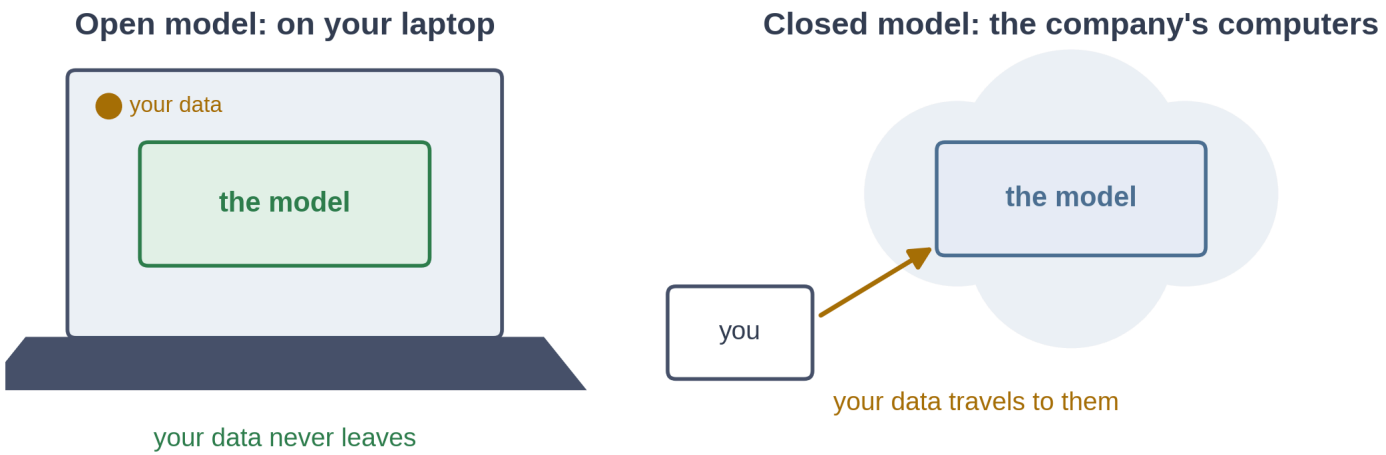
The same models, on the same three-step sums. Only the scoring differs. (Illustration.)

KEY IDEA

When you read that a model “suddenly” gained a skill, ask how the skill was measured. That is a Week 1 question.

RUN IT YOURSELF, OR RENT IT ONLINE

Open models and closed models



KEY IDEA

Open (Llama, DeepSeek): free to download, private, yours to change; small ones are weaker. Closed (ChatGPT): usually stronger, but online, and your data goes to the company. Choose by the job.

LIVE DEMO · 3 MINUTES · OLLAMA

A small open model, on a laptop

Command Prompt (Windows) or Terminal (Mac)

```
ollama run llama3.2:1b
>>> In two sentences, why does a sourdough starter rise?
```

1.3 GB

download, once

offline

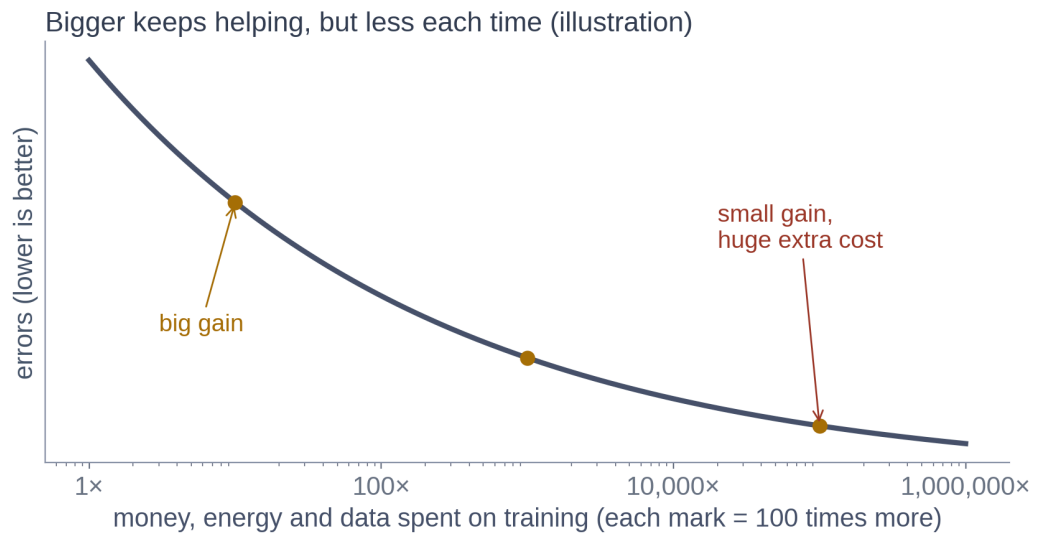
runs with no internet

KEY IDEA

Ollama is a free program that runs open models on your own computer. You will install it at home; the steps are at the end of today and in your booklet.

SCALING, AND WHERE IT STRAINS

Bigger has worked, but each step gains less



KEY IDEA

For years: more data + more computing + a bigger model = a better model. Now each big step up gives a **smaller** gain, at a **much higher** cost.

Four walls



KEY IDEA

Cost: one top model is reported to cost over $100 million to train. Energy: on the scale of power stations. Data: good new text is running out. Smaller gains: each step up helps less. Weeks 8 and 9 look at **faster** and **smaller** models.

KEY POINTS

Part 1, done

★ KEY POINTS TO REMEMBER

- An LLM guesses the next word (strictly, the next token), one at a time, extremely well.
- Inside is a transformer: attention lets every word look at every other word.
- It is made in three stages: pretraining, fine-tuning, and learning from people’s feedback.
- Open models run on your computer; closed models run online. Choose by the job.
- Bigger has worked, but cost, energy, data and smaller gains are now walls.

PART 2

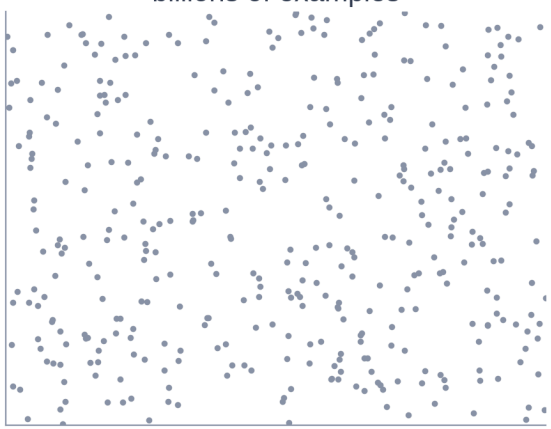
How it connects to your project

From billions of examples to one law

DATA, OR A LAW

Two opposite ways to learn

A language model learns from billions of examples



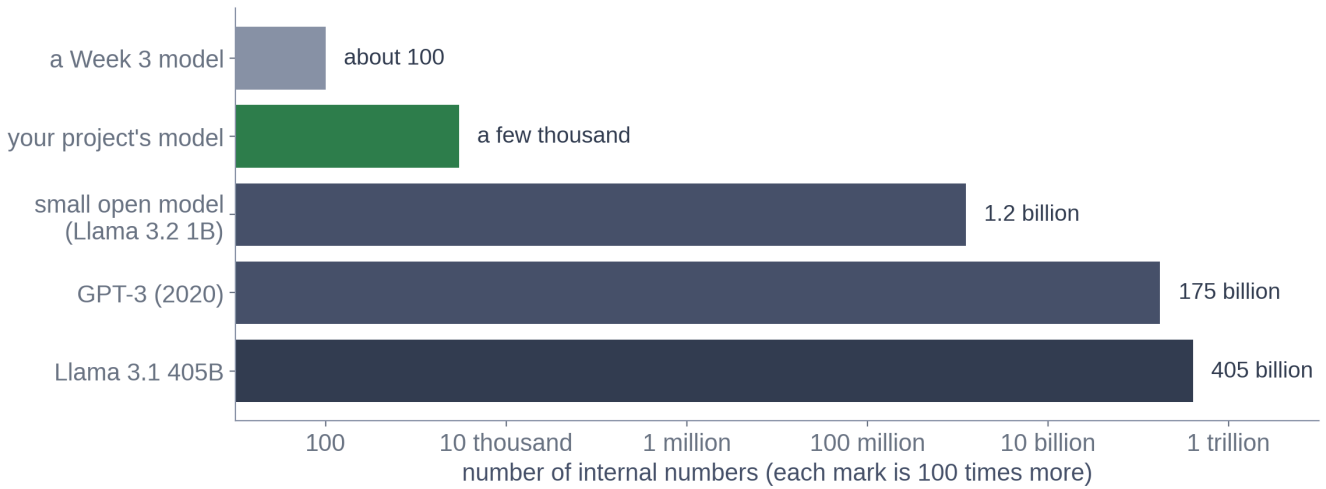
Your project learns from one law of science, no data

how fast it changes = the law

KEY IDEA

A language model learns everything from data. Your project learns from a law of science, with no data at all. When data is scarce or expensive, a law can do what data cannot.

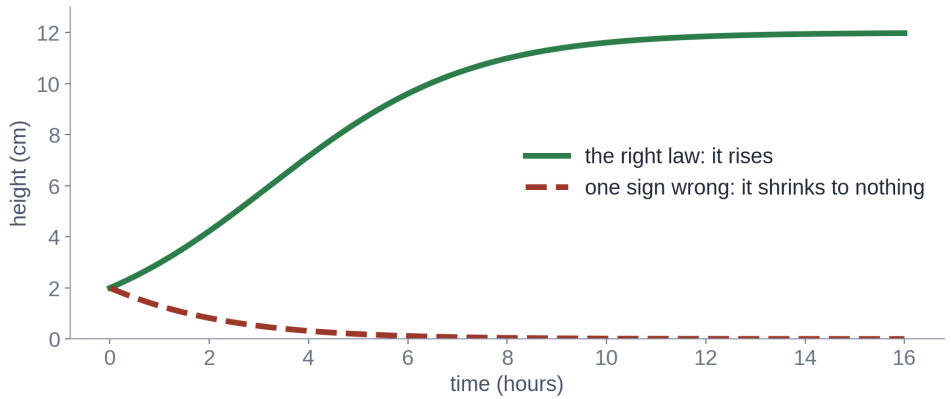
Why this matters right now



KEY IDEA

Your model, in green: a few thousand internal numbers, not billions. It trains in a minute or two and needs no data, which is exactly why models that know a law are interesting now.

Use a language model as a helper, then check it



Good uses

Explain your law in plain words. Explain an error message. Suggest where a number might come from.

Then check

Compare with your student guide or a textbook. One wrong sign and the answer goes the wrong way.

FOUR THINGS, WRITTEN SO A COMPUTER CAN USE THEM

This week’s project step

Problem card · class example: a sourdough starter

1 · The law	how fast it rises = r × height × (1 − height ÷ top of jar)
2 · The numbers	r = 0.5 per hour (a baking book); K = 12 cm (measured the jar)
3 · The start	height = 2 cm at time 0; look from 0 to 16 hours
4 · The answer sheet	exact formula and solver, both plotted: they agree

1 · The law, as one line of code

how fast it rises = r × height × (1 − height ÷ top of the jar)

In words: it rises fast at first, then slows down near the top of the jar.

capstone_step1.ipynb python

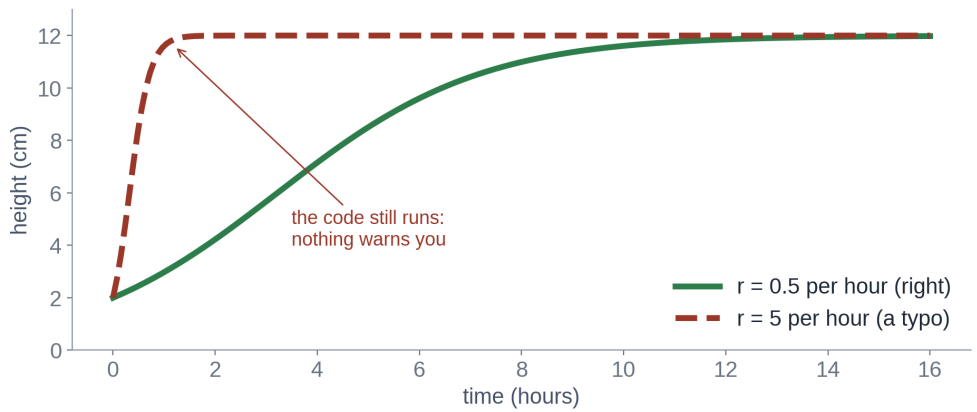
```python
def rate(t, H):                      # how fast the height changes
    return r * H * (1 - H / K)       # <- the law, as one line of code
```

KEY IDEA

Every law in the student guide can be said as "how fast it changes = …". Say yours in words first; then the code line copies the words.

2 · The numbers, and 3 · the start

what	value	where from
growth rate r	0.5 per hour	a baking book
top of jar K	12 cm	measured
start H0	**2 cm**	**measured**

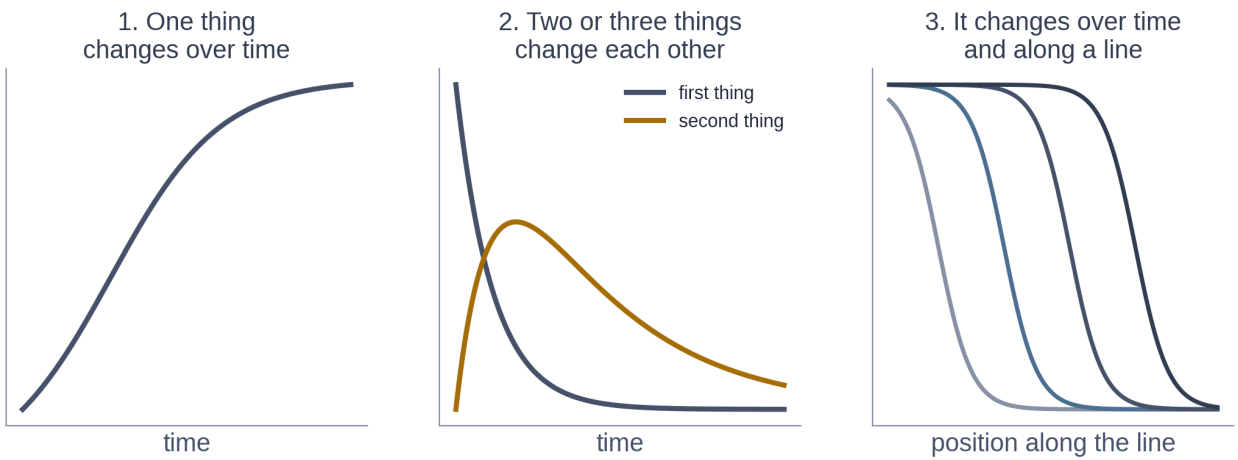


KEY IDEA

Write down where every number came from. Keep numbers close to 1 with sensible units: hours rather than seconds. A typo in a number does not stop the code; it just answers the wrong question.

IT DECIDES WHICH PART OF THE NOTEBOOK YOU USE

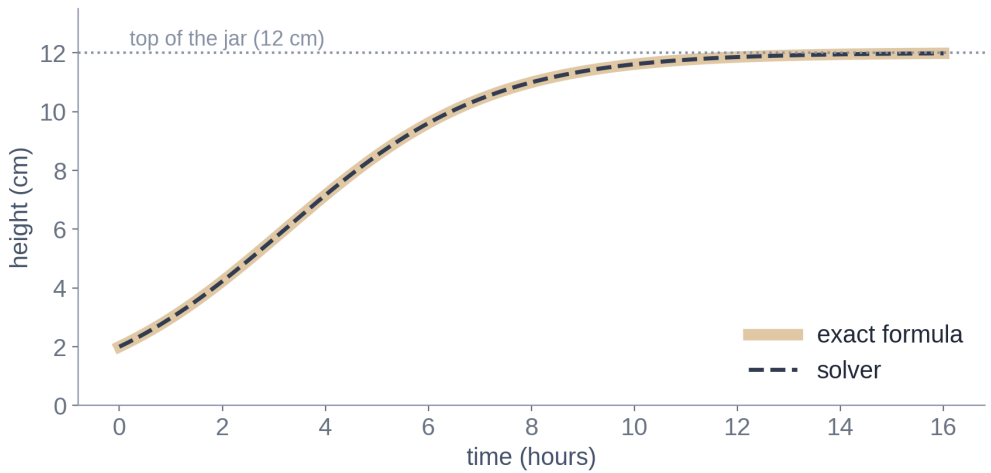
Which shape is your law?



KEY IDEA

Your idea form, question 18: **small** means shape 1 (Part 2 of the notebook), **medium** means shape 2 (Part 3), **large** means shape 3 (Part 4).

4 · The answer sheet



KEY IDEA

Two ways to make it: an **exact formula**, if your law has one, or a **solver**, which works the answer out in many small steps. If you can, make both: they should lie on top of each other.

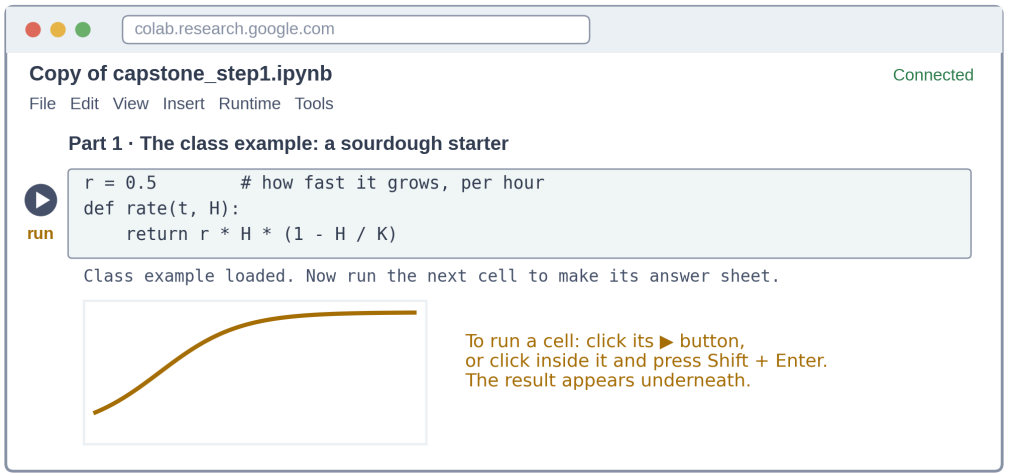
PART 3

Lab walkthrough

This week’s step, live in Google Colab · 20 minutes

PYTHON IN YOUR WEB BROWSER, NOTHING TO INSTALL

Google Colab, in one slide



Free

You only need a Google account.

In the browser

Nothing to install on your laptop.

Notebooks

Text and code in one page, in cells.

READING, NOT WRITING

The Python you need today: five things

you will see	it means
`# a note`	a note for humans; the computer ignores it
`r = 0.5`	store the number 0.5 under the name r
`def rate(t, H):`	a rule called rate, which needs t and H
`return …`	what the rule gives back (the indent matters)
`* / ** np.sqrt np.exp`	times, divide, to the power of, square root, e to the power of

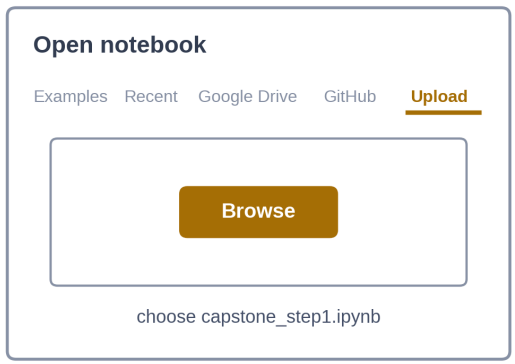
KEY IDEA

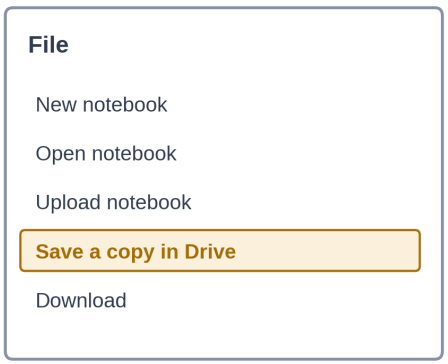
Red text after you run a cell is an error message. Read its **last line** it is usually a typo or a missing bracket.

ABOUT 4 MINUTES

Steps 1 and 2: open the notebook and save a copy

1. Download the Week 7 lab file from Tutors and unzip it: you get `capstone_step1.ipynb`.
2. Go to **colab.research.google.com** and sign in.
3. Choose **Upload**, then **Browse**, and pick the file.
4. Click **File → Save a copy in Drive**, and work in the copy.





IF YOU SEE A WARNING

"This notebook was not authored by Google": click **Run anyway**. It is our notebook from Tutors.

ABOUT 4 MINUTES

Step 3: run Step 0 and the class example

capstone_step1.ipynb · Part 1 python

```python
r     = 0.5     # how fast it grows, per hour   (source: a baking book)
K     = 12.0    # the top of the jar, in cm     (source: I measured the jar)
H0    = 2.0     # the starting height, in cm
t_end = 16.0    # look from 0 to 16 hours

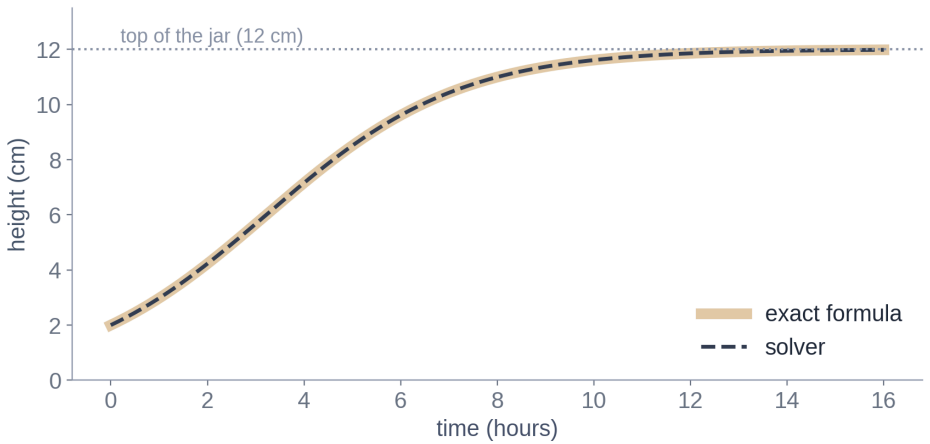
def rate(t, H):                        # how fast the height changes
    return r * H * (1 - H / K)         # <- THE LAW, as one line of code
```

KEY IDEA

Run cells **in order**, top to bottom. The first time, Colab takes a few seconds to connect.

ABOUT 4 MINUTES

Step 4: make the answer sheet



what the cell prints

Biggest difference between formula and solver: 0.0 cm
Biggest leftover when the answer is put back into the law: 0.0112

ABOUT 6 MINUTES · WHERE YOUR OWN LAW GOES

Step 5: the YOUR TURN cell

capstone_step1.ipynb · Part 2 · placeholder law: a tank draining — python

```python
name  = "water height (cm)"     # <-- what changes, and its unit
a     = 0.5                     # <-- a number   (unit, source)
start = 25.0                    # <-- the starting value
t_end = 15.0                    # <-- how long to look

def my_rate(t, y):
    return -a * np.sqrt(y)      # <-- YOUR LAW: how fast it changes

HAS_EXACT = True                # <-- does your law have an exact formula?
```

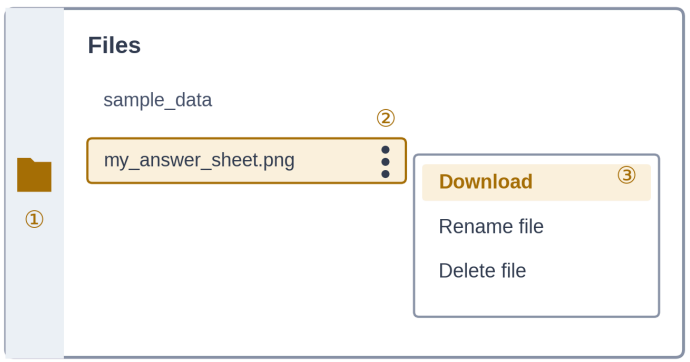
KEY IDEA

At home, replace every line marked `<--` with your own law.

ABOUT 2 MINUTES

Step 6: save the picture, and the other shapes

1. Click the **folder icon** on the left of Colab.
2. Find `my_answer_sheet.png` and click its **three dots**.
3. Click **Download**.



Part 3 of the notebook

If two or three things change each other. Template: two connected water tanks.

Part 4 of the notebook

If it changes over time and along a line. Template: a species spreading along a coast.

1 HOUR AT MOST · DUE BEFORE WEEK 8

Your lab at home

Part A · 15 min · the topic

Install Ollama and run a small model. Count the tokens in one sentence. Ask Ollama and ChatGPT to explain your law: were they right?

Part B · 45 min · your project

Do Steps 1 to 6 with your own law. Hand in: your law line, your numbers and where you found them, your start, and your answer-sheet picture.

KEY IDEA

Hand in on the Week 7 form on Tutors. Stuck for 40 minutes? Stop and post in the channel.

ABOUT 10 MINUTES, MOSTLY WAITING FOR THE DOWNLOAD

Part A at home: Ollama, step by step

Windows

1. Go to **ollama.com**, click **Download**, choose Windows, run the file.
2. Press Start, type `cmd`, press Enter.
3. Type `ollama run llama3.2:1b` and press Enter (1.3 GB, once).
4. At `>>>`, type your question. Type `/bye` to leave.

Mac

1. Go to **ollama.com**, click **Download**, choose macOS, open the file, move Ollama to Applications, open it.
2. Press Cmd + Space, type `Terminal`, press Enter.
3. Type `ollama run llama3.2:1b` and press Enter (1.3 GB, once).
4. At `>>>`, type your question. Type `/bye` to leave.

> **KEY IDEA**
>
> Old or small laptop? Use `ollama run gemma3:1b` instead (815 MB). Tokens: go to **platform.openai.com/tokenizer** and type a sentence.

KEY POINTS

This week, in five lines

★ KEY POINTS TO REMEMBER

- An LLM guesses the next token, using a transformer, trained in three stages.
- Open models run on your computer; closed models run online.
- Scaling is hitting walls: cost, energy, data and smaller gains.
- Your project learns from a law, not data: tiny, cheap, and no data needed.
- This week: your law as one line of code, its numbers, its start, and its answer sheet.

Billions of examples, or one good law.

This week, write yours down.

Questions any time in the channel.